Word2vec与Bert微调文本分类实验

马思远[1)]

[1)](南开大学密码与网络空间安全学院, 天津, 中国)

摘 要 本文围绕新闻文本分类任务，研究并比较不同文本表示方法的分类效果。实验以New York Times（NYT）新闻数据集作为主要分类数据集，按照80%、10%、10%的比例划分训练集、验证集和测试集。首先，采用Binary Bag of Words和Word Frequency方法构建文本特征，并使用Logistic Regression进行分类。随后，分别使用预训练GloVe词向量、基于AG News训练的Word2Vec以及基于NYT训练的Word2Vec，通过平均词向量生成100维文档表示，并使用Logistic Regression完成分类。最后，利用预训练BERT-base-uncased模型进行3轮Fine-tuning，实现端到端文本分类。实验统一采用Accuracy和Macro-F1作为评价指标。结果表明，BERT模型在NYT测试集上取得了98.00%的Accuracy和95.01%的Macro-F1，整体表现优于本次实验中的其他文本表示方法。

关键词 文本分类；Bag of Words；GloVe；Word2Vec；BERT；Fine-tuning

Text Classification Using Word2Vec and BERT Fine-Tuning

Siyuan-Ma[1)]

[1)](College of Cryptology and Cyber Science, Nankai University, Tianjin, China)

Abstract This report investigates and compares different text representation methods for news text classification. The New York Times (NYT) dataset is used as the primary classification dataset, with 80%, 10%, and 10% of the data allocated to training, validation, and test sets, respectively. First, Binary Bag of Words and Word Frequency representations are evaluated using Logistic Regression. Next, three embedding-based approaches are investigated: pre-trained GloVe embeddings, Word2Vec trained on AG News, and Word2Vec trained on NYT. For these approaches, document representations are obtained by averaging 100-dimensional word vectors, followed by Logistic Regression classification. Finally, the pre-trained BERT-base-uncased model is fine-tuned for three epochs to perform news classification. All models are evaluated using Accuracy and Macro-F1. The experimental results show that BERT achieves an Accuracy of 98.00% and a Macro-F1 of 95.01% on the NYT test set, outperforming the three embedding-based approaches in this experiment.

Keywords Text Classification; Bag of Words; GloVe; Word2Vec; BERT; Fine-tuning

1 引言

不同的文本表示方法具有不同的特点。传统的Bag of Words方法通过统计词语是否出现或出现的频率构建文本特征，具有实现简单、容易理解等优势，但难以直接表达词语之间的语义关系。Word Embedding方法将单词映射到连续的低维向量空间，其中Word2Vec和GloVe是具有代表性的词向量方法。相比之下，基于Transformer架构的BERT预训练语言模型能够结合上下文生成文本表示，并通过Fine-tuning适应具体的分类任务。

实验以NYT新闻数据集为主要研究对象，分别实现并比较传统Bag of Words、预训练GloVe、基于AG News和NYT训练的Word2Vec，以及BERT Fine-tuning等文本分类方法。其中，Bag of Words和词向量方法统一使用Logistic Regression分类器，BERT则采用预训练模型进行端到端微调。所有实验使用相同的NYT数据划分，并通过Accuracy和Macro-F1评价模型性能。通过比较不同方法的实验结果，分析文本表示方式及训练语料对新闻分类效果的影响。

2 实验设置

2.1 数据集划分

实验以NYT新闻数据集进行文本分类。为保证不同文本表示方法之间的实验结果具有可比性，所有分类实验均使用相同的数据划分。首先删除文本或标签缺失的样本，随后采用Scikit-learn中的train_test_split函数，在random_state=42和shuffle=True的设置下随机划分数据集。

具体而言，首先将80%的数据作为训练集，其余20%作为临时数据集，再将临时数据集平均划分为验证集与测试集，最终得到训练集80%、验证集10%、测试集10%的划分比例。实际划分结果如表1所示。

Table 1 NYT数据集划分结果

数据集	比例	样本数量
Training Set	80%	9215
Validation Set	10%	1152
Test Set	10%	1152
总计	100%	11519

训练集用于学习模型参数，验证集用于模型选择及超参数调整，测试集仅用于最终性能评价。所有实验统一在NYT Test Set上报告最终分类结果。

2.2 评价指标

实验统一使用Accuracy和Macro-F1评价文本分类模型的性能。

Accuracy（准确率）表示正确分类的样本数量占测试集总样本数量的比例，计算公式如下：

$$\mathrm{Accuracy} = \frac{N_{\mathrm{correct}}}{N_{\mathrm{total}}} \tag{1}$$

其中，N_{correct}表示预测正确的样本数量，N_{total}表示测试样本总数。

Macro-F1首先分别计算每个类别的Precision、Recall和F1-score，再对所有类别的F1-score取算术平均：

$$F1_k = \frac{2P_kR_k}{P_k + R_k} \tag{2}$$

$$\text{Macro-F1} = \frac{1}{K}\sum_{k=1}^{K} F1_k \tag{3}$$

其中，P_k和R_k分别表示第k类的Precision和Recall，K表示类别数量，本实验中$K = 3$。当分母为零时，对应F1值按零处理。

由于Macro-F1对每个类别赋予相同权重，因此能够更加全面地反映模型在不同类别上的分类性能。

2.3 实验环境与依赖

本实验在Windows操作系统下完成，使用Visual Studio Code（VS Code）作为主要代码编辑与运行工具，使用Python编写程序，并通过Anaconda管理不同实验所需的Python环境。

Task 1与Task 2主要使用Anaconda原有环境，完成Bag of Words、GloVe、Word2Vec和Logistic Regression实验；Task 3使用独立的nlp-bert环境，Python版本为3.11，采用CPU版本的PyTorch对BERT进行Fine-tuning。

本实验使用Pandas与NumPy进行数据读取及向量计算，使用NLTK对新闻文本分词。使用Scikit-learn完成数据集划分与逻辑回归分类，并通过Gensim训练Word2Vec词向量。BERT模型使用PyTorch与Transformers进行微调与测试。

标准库包括pathlib、time和math，评价模块是自行编写的accuracyandmacro.py。上述文件都不需要通过pip单独安装。

为方便实验复现，下面列出各环境所需的依赖安装命令。

（**1**）**Task 1**与**Task 2**依赖安装

```
conda activate base
python -m pip install pandas
python -m pip install numpy
python -m pip install scikit-learn
python -m pip install nltk
python -m pip install gensim
```

（**2**）**Task 3**依赖安装

```
conda activate nlp-bert
python -m pip install pandas
python -m pip install numpy
python -m pip install scikit-learn
python -m pip install transformers
python -m pip install accelerate
```

Task 3使用PyTorch官方CPU安装源，安装命令为：

```
python -m pip install torch --index-url
https://download.pytorch.org/whl/cpu
```

3 文本分类方法与实现

3.1 GloVe与Word2Vec词向量方法

实验统一使用100维词向量，并采用平均词向量的方法构建文档表示。

第一组方法使用公开的预训练GloVe词向量。实验加载glove.6B.100d文件，根据分词结果查找对应的单词向量。由于词向量已经预先训练完成，因此不需要重新训练词嵌入模型。

第二组方法使用AG News语料训练Word2Vec。实验通过Gensim实现Skip-gram模型，设置向量维度为100，窗口大小为5，最低词频为2，训练轮数为5，并固定随机种子为42。训练完成后，使用得到的词向量表示NYT新闻文本。

第三组方法使用NYT训练集训练Word2Vec。训练参数与第二组相同，但词向量仅使用NYT训练集学习，验证集和测试集不参与词向量训练。通过比较两组Word2Vec结果，可以分析不同训练语料对新闻分类效果的影响。

对于上述三种方法，首先对文档进行分词，然后查找词表中存在的有效词向量，并计算其平均值。文档向量的计算公式如下：

$$\mathbf{e}(d) = \frac{1}{n}\sum_{i=1}^{n}\mathbf{e}(w_i) \tag{4}$$

最后，将得到的文档向量输入逻辑回归分类器。实验分别测试正则化参数C取0.1、1和10时的效果，并根据验证集的Macro-F1选择分类器，最终在NYT测试集上进行评价。

3.2 BERT模型微调方法

实验使用Transformers库加载预训练模型及对应的分词器，并将输出类别数量设置为3，分别对应business、politics和sports。对于输入文本，分词器将其转换为词元编号及注意力掩码。最大序列长度设置为64，超过长度的文本进行截断，不足长度的文本通过填充保持统一尺寸。

训练阶段使用PyTorch实现参数更新，采用AdamW优化器，学习率设置为2×10^{-5}，批次大小为8，共训练3轮。由于运行设备为CPU，模型训练在独立的Python环境中完成。训练过程中，根据分类损失计算梯度，并通过反向传播更新模型参数。

每轮训练完成后，实验在验证集上计算Accuracy和Macro-F1，以验证集Macro-F1作为最佳模型的选择依据。当模型表现优于此前轮次时，保存当前模型参数及分词器。三轮训练结束后，使用保存的最佳模型在独立的NYT测试集上进行预测，并计算最终分类指标。

4 实验项目结构与核心代码

4.1 实验项目结构

实验采用分任务组织代码的方式，将数据文件、词向量模型、训练结果和Python程序分别存放。项目中与实验复现相关的主要文件结构如下，省略自动缓存及编辑器配置文件。

```
NLP-task1/
  HW-1/
    nyt.csv
    ag.csv
  glove/
    glove.6B.100d.txt
  models/
    ag_word2vec.model
    nyt_word2vec.model
    bert_best/
  result/
    task2_results.txt
    task3-result.txt
  accuracyandmacro.py
  task1.py
  task2.py
  task2-2.py
  task2-3.py
  task3.py
  task3-test.py
```

其中，task1.py用于词袋模型分类；task2.py用于预训练GloVe实验；task2-2.py和task2-3.py分别负责两组Word2Vec训练；task3.py用于BERT微调，task3-test.py用于加载最佳模型并完成最终测试。accuracyandmacro.py提供统一的评价函数。

4.2 核心代码实现

词袋模型通过CountVectorizer构建文本特征，并使用逻辑回归进行分类。二值词袋的核心代码如下。词频表示采用相同流程，只需将binary参数改为False。

```
binaryvectorizer = CountVectorizer(
    tokenizer=tokenize_text,
    token_pattern=None,
    binary=True
)
trainbinary = binaryvectorizer.fit_transform(
    traintexts)
binarymodel = LogisticRegression(
    max_iter=1000)
binarymodel.fit(trainbinary, trainlabels)
```

词向量方法的关键是将单词向量转换为文档向量。实验查找文档中具有有效词向量的单词，对其向量取平均；若没有有效词向量，则使用零向量。核心逻辑简化如下：

```
vectors = [word_vectors[w] for w in tokens
           if w in word_vectors]
doc_vec = (np.mean(vectors, axis=0)
           if vectors else np.zeros(100))
```

两组Word2Vec使用相同的Skip-gram训练参数，主要区别在于输入语料。AG News实验使用ag_tokens，NYT实验使用train_tokens。以下以NYT训练为例：

```
nyt_word2vec = Word2Vec(
    sentences=train_tokens,
    vector_size=100,
    window=5, min_count=2,
    sg=1, epochs=5,
    workers=1, seed=42
)
```

BERT微调通过预训练分词器将文本编码为固定长度的输入序列，再使用AdamW优化器更新模型参数。核心设置如下：

```
def encode_texts(texts):
    return tokenizer(
        texts.tolist(),
        padding="max_length",
        truncation=True,
        max_length=64,
        return_tensors="pt"
    )

optimizer = torch.optim.AdamW(
    model.parameters(), lr=2e-5)
```

BERT训练循环共执行3轮，每轮结束后计算验证集指标，并保存Macro-F1最高的模型。最终通过独立测试程序加载最佳模型，完成测试集评价。以上仅列出关键语句，完整训练与测试流程见对应Python文件。

Table 2 词向量方法在NYT测试集上的分类结果

方法	Accuracy	Macro-F1
GloVe	0.9774	0.9403
Word2Vec-AG	0.9722	0.9251
Word2Vec-NYT	0.9792	0.9469

5 实验结果与分析

5.1 词向量方法实验结果

分类结果如表2所示。

三种词向量方法均取得了较高的分类准确率。其中，基于NYT训练的Word2Vec表现最好，

两组Word2Vec方法使用相同的训练参数。

预训练GloVe的Accuracy为97.74%，Macro-F1为94.03%，同样表现良好。这表明公开的通用词向量能够为新闻文本分类提供有效的语义特征，但在当前数据划分下，其表现略低于使用NYT训练集学习的词向量。

5.2 BERT微调实验结果

实验使用预训练BERT模型进行新闻分类，设置最大输入长度为64，批次大小为8，学习率为2×10^{-5}，训练轮数为3。每轮训练结束后，记录平均训练损失，并在验证集上计算Accuracy和Macro-F1。具体结果如表3所示。

Table 3 BERT训练与验证结果

Epoch	Loss	Accuracy	Macro-F1
1	0.1279	0.9688	0.9361
2	0.0382	0.9627	0.9274
3	0.0225	0.9714	0.9411

表3中的Loss表示平均训练损失，Accuracy和Macro-F1为验证集指标。随着训练轮数增加，平均训练损失持续下降，说明模型逐步拟合训练数据。但验证集性能并未单调提高，第二轮的评价指标略有下降，第三轮则达到最高水平。

第三轮的验证集Accuracy为97.14%，Macro-F1为94.11%。根据验证集Macro-F1选择最佳模型，因此实验保存第三轮训练得到的模型参数和分词器，用于最终测试。

实验在CPU环境下完成全部三轮BERT微调，实际耗时约1小时47分钟。完成模型选择后，实验加载保存的最佳模型，在独立的NYT测试集上进行预测。最终结果如表4所示。

最终，BERT在测试集上取得98.00%的Accuracy和95.01%的Macro-F1。 与

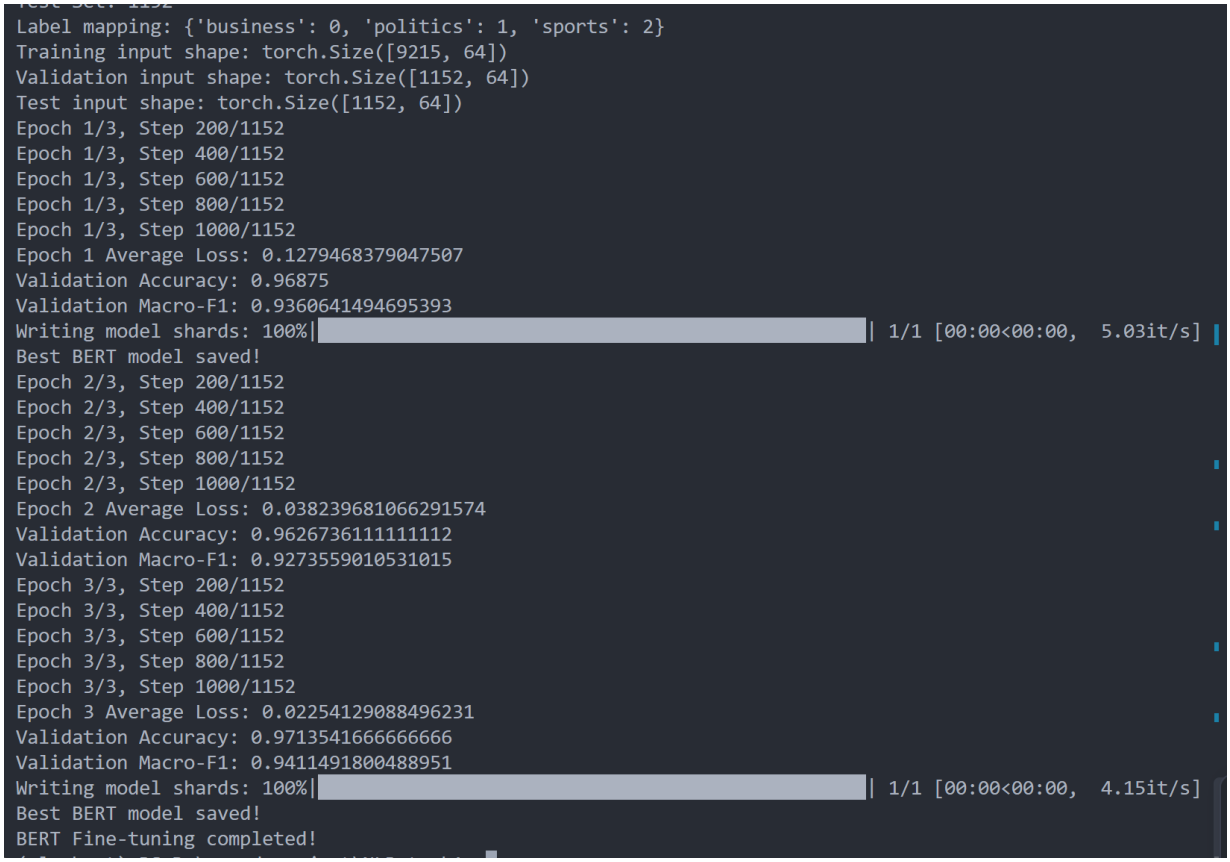


Fig. 1 BERT模型三轮微调的训练日志与验证集结果

Table 4 BERT在NYT测试集上的分类结果

评价指标	测试结果
Accuracy	0.9800
Macro-F1	0.9501

前述三种词向量方法相比，BERT获得了更高的分类指标，说明经过针对性微调的上下文表示能够有效完成新闻文本分类任务。

6 总结

对于BERT，则通过微调预训练模型学习分类任务所需的特征。在统一的数据划分和评价标准下，NYT训练的Word2Vec取得了97.92%的准确率和94.69%的Macro-F1，BERT取得了98.00%的准确率和95.01%的Macro-F1。这些结果表明，不同文本表示方式均能够提取有效的新闻分类特征，但模型复杂度的增加并不一定带来大幅度的性能提升。

Word2Vec和GloVe将单词映射到低维连续向量空间，使语义相关的词语能够具有相近的表示。但Word2Vec生成的是静态词向量，同一个词在不同上下文中使用相同的表示。简单平均会弱化词序信息，也无法区分不同单词对分类任务的重要程度。因此，这种方法虽然计算过程相对简单，却不能充分利用完整的句子结构。

BERT的合理性在于，它能够结合上下文生成文本表示，并通过微调使预训练参数适应新闻分类任务。但这种优势需要付出计算代价。实验在CPU环境下完成三轮微调耗时约1小时47分钟，而BERT相对于NYT词向量方法的准确率提升不足0.1个百分点，Macro-F1提升约0.32个百分点。

实验仍存在一定局限。平均词向量没有考虑词语的重要程度，而BERT最大输入长度设置为64，较长新闻的后续内容可能被截断。后续可以尝试加权词向量、不同的输入长度以及多随机种子重复实验，进一步分析各方法的性能变化。总体而言，实验说明文本分类方法的选择需要兼顾表示能力、训练数据的适配程度和计算开销。对于类别特征明显的新闻分类任务，简单方法可能已经具有较强竞争力；对于更依赖上下文语义的任务，预训练语言模型则可能体现出更大的优势。